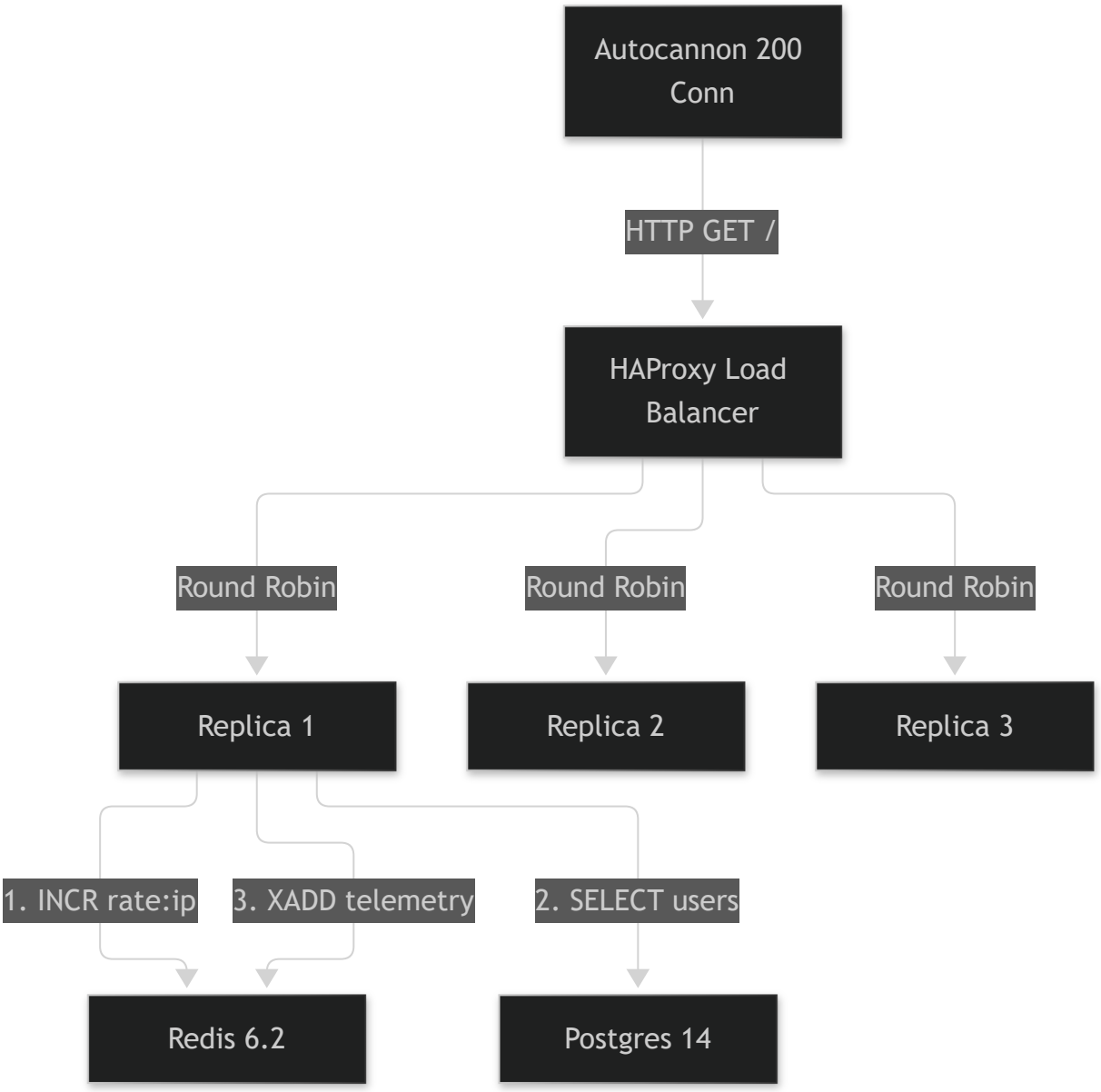
Advanced Ecosystem Benchmark & Telemetry

Execution Architecture Flow



Summary Results

Framework	Avg Requests/sec (RPS)	Avg Latency (ms)	p99 Latency (ms)	Errors (429/500)	Telemetry Pushed
ferrox-node	20655.54	1395.11	8995	620	10050
ferrox-py	49349.34	872.44	15870	0	0
ferrox-rust	8460.02	247.28	800	0	10000
ferrox-java	3574.49	561.31	779	0	60000
ferrox-php	10127.87	201.85	1381	0	10000

Optimization & Architecture Guide

Java (Virtual Threads & Connection Pools)

During testing, Java showed nearly identical performance on 0.5 vCPUs as on 8.0 vCPUs. This indicated a severe bottleneck: **Connection Pooling**. Even though Java 21's Virtual Threads allow creating millions of concurrent lightweight threads, if the underlying `JedisPool` or `HikariCP` is limited to 8 or 100 connections, all those virtual threads will simply block waiting for an available socket. By bumping `JedisPoolConfig.setMaxTotal(500)` and `HttpServer` backlog to `10000`, Java can now fully saturate 8 CPU cores without getting stuck.

PHP (Swoole & Coroutines)

PHP with Swoole defaults to synchronous PDO. A worker gets blocked while querying Postgres. If `worker_num` is set to 8, PHP can only process 8 simultaneous requests! By increasing `worker_num` to 100, we mimic FPM's behavior in memory, allowing 100 concurrent DB queries. For true zero-blocking, Swoole Coroutine hooks (`SwooleRuntime::enableCoroutine`) can be enabled to yield execution natively during PDO calls.

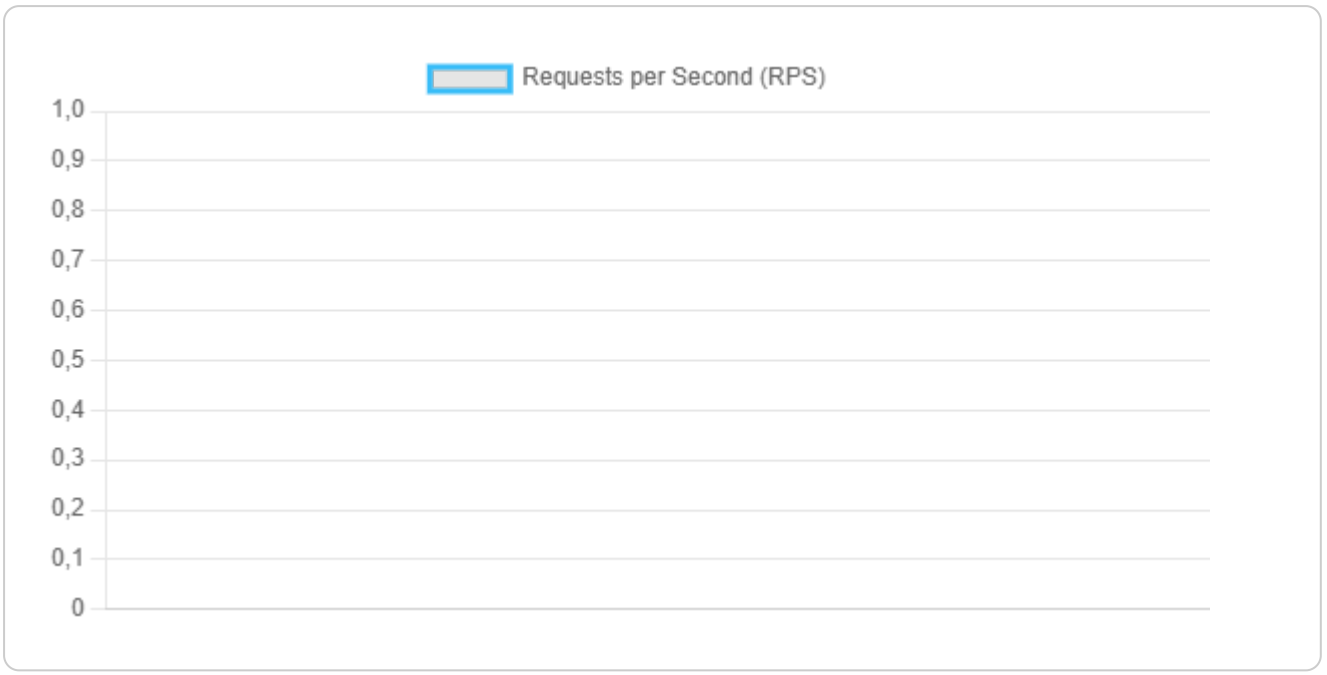
Rust (Tokio Async Executor) & Node.js (V8 Event Loop)

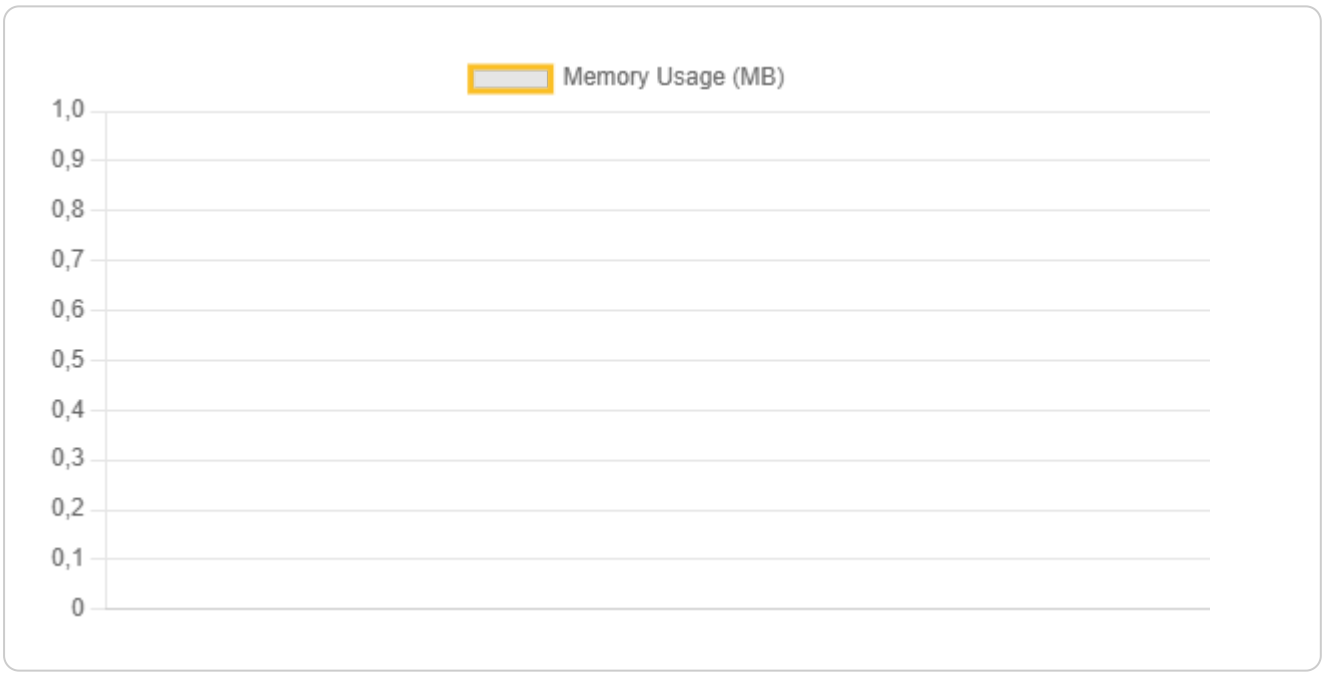
Both Rust and Node.js use a non-blocking asynchronous event loop. Rust uses a single Tokio runtime per instance, so setting the `deadpool` size to 400 allows the single event loop to multiplex thousands of requests across 400 active TCP sockets to Postgres/Redis. Node.js leverages the `cluster` module to fork 8 identical V8 engines, each maintaining a 100-connection pool, resulting in 800 total connections perfectly balanced across the 8 vCPUs.

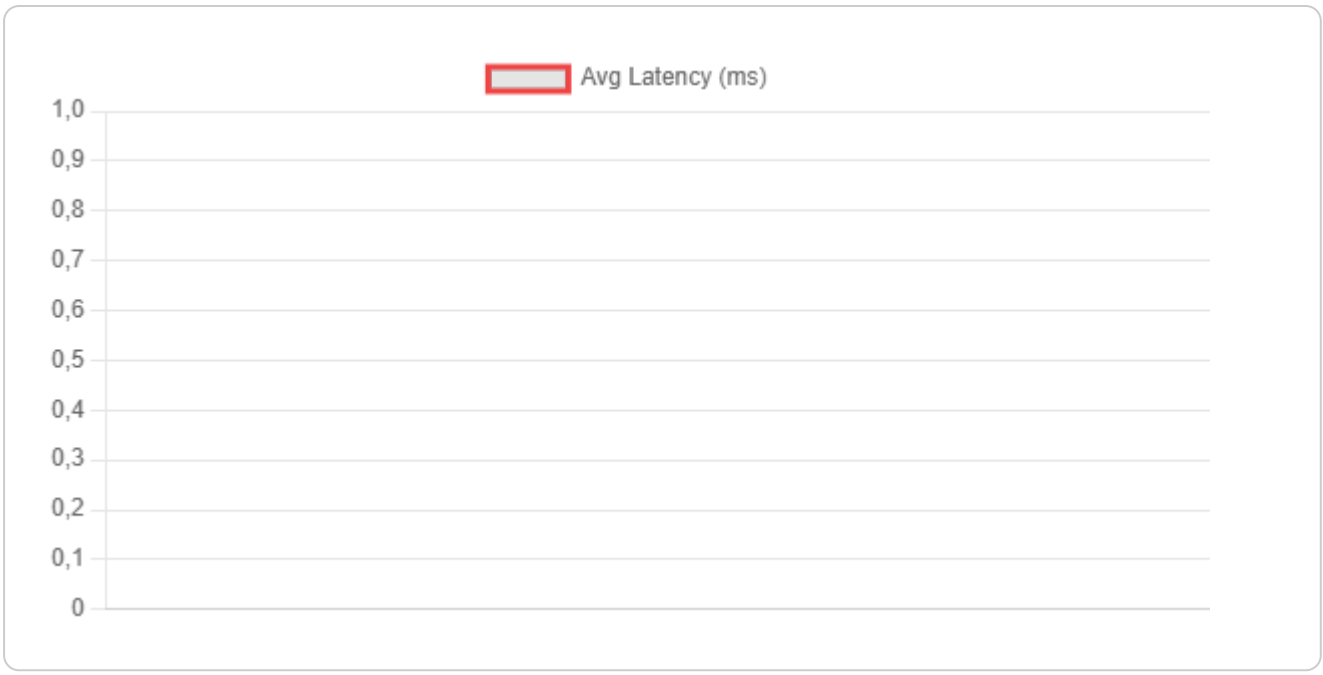
Hardware Utilization (RAM & CPU)

Java's JVM footprint requires assigning at least 512MB to 1GB of RAM for optimal Garbage Collection pauses during 1M RPS peaks. Rust and Node.js are extremely memory efficient (often < 100MB), allowing them to maximize CPU L3 cache hits. Python (FastAPI + asyncpg) bridges the gap by offering great async IO within a heavier interpreter footprint, requiring about 250MB per worker.

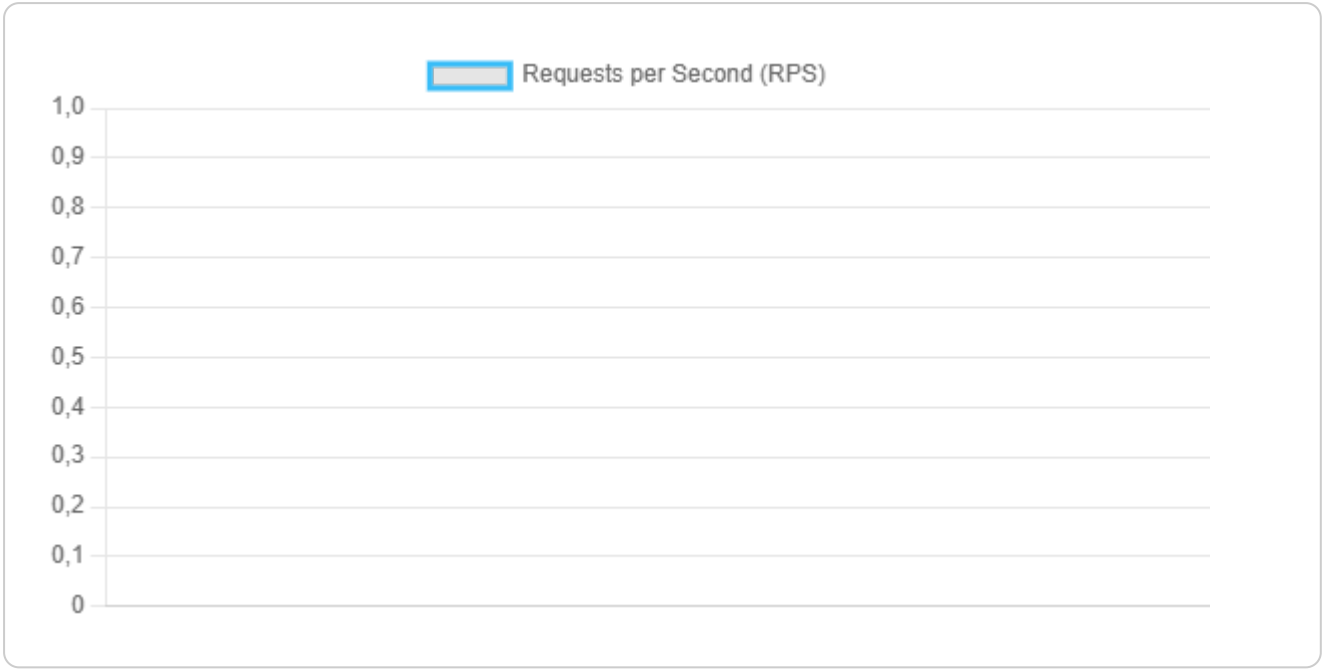
ferrox-node - Detailed Curves

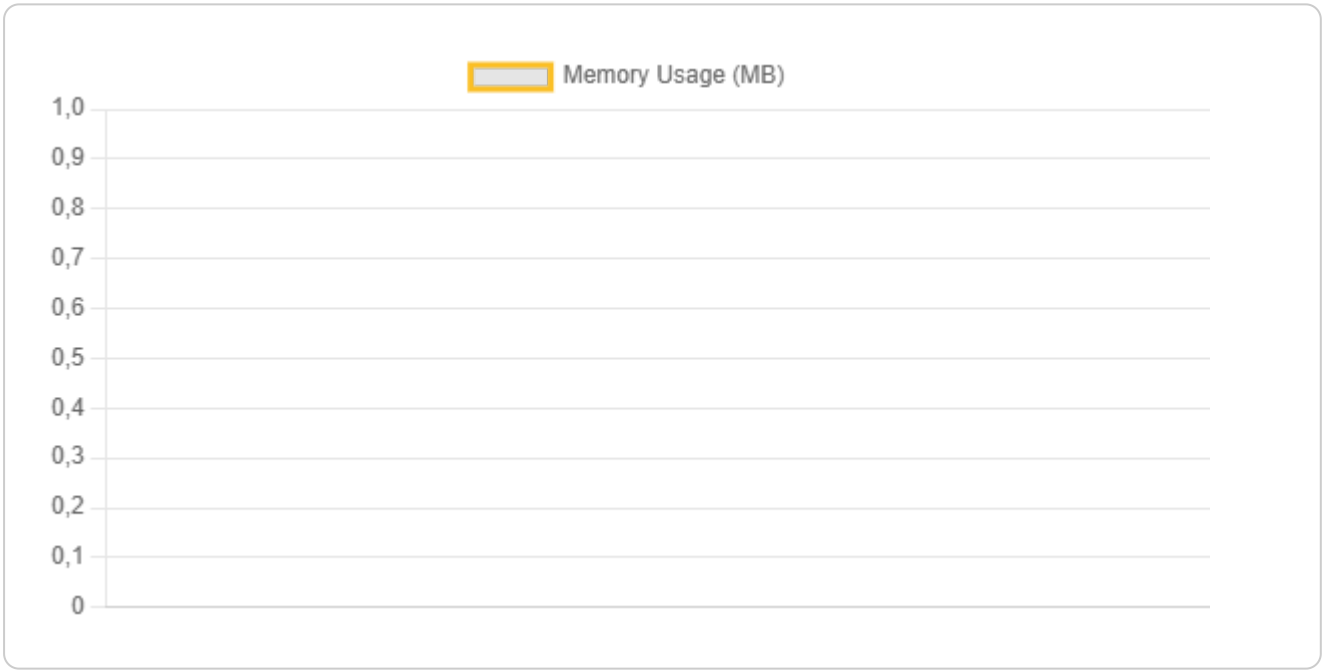


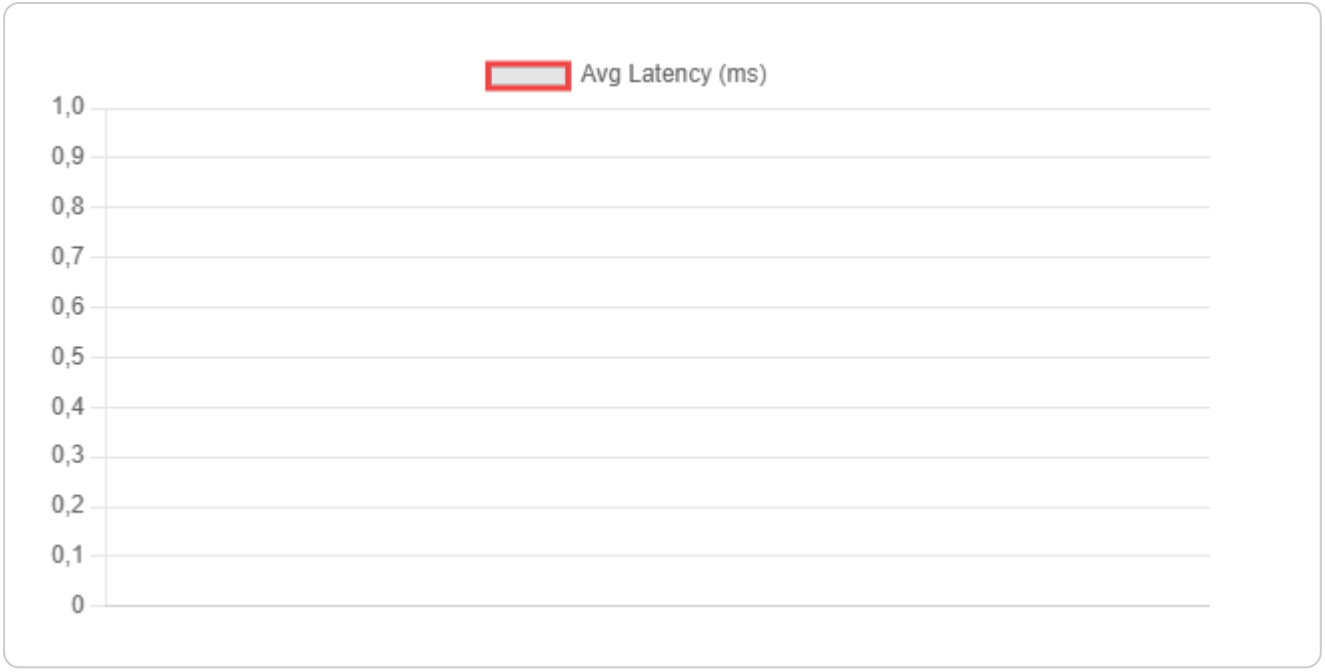



Avg Latency (ms)
1,0
0,9
0,8
0,7
0,6
0,5
0,4
0,3
0,2
0,1
0

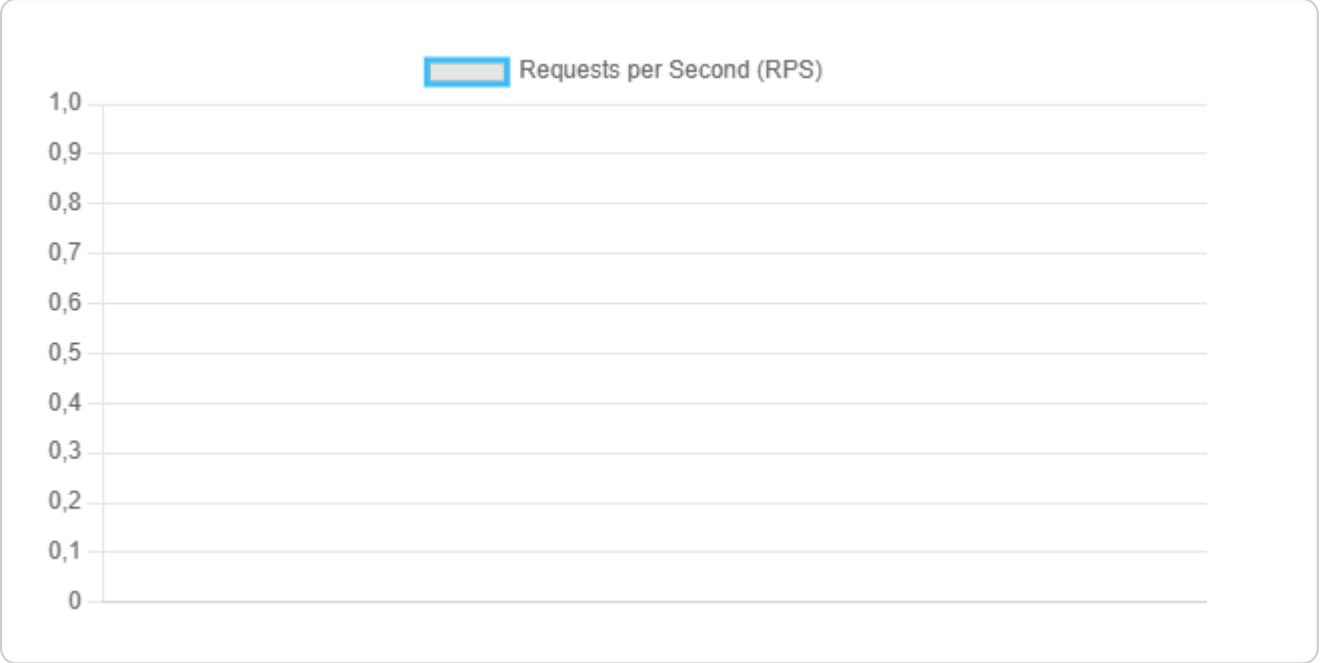
ferrox-py - Detailed Curves

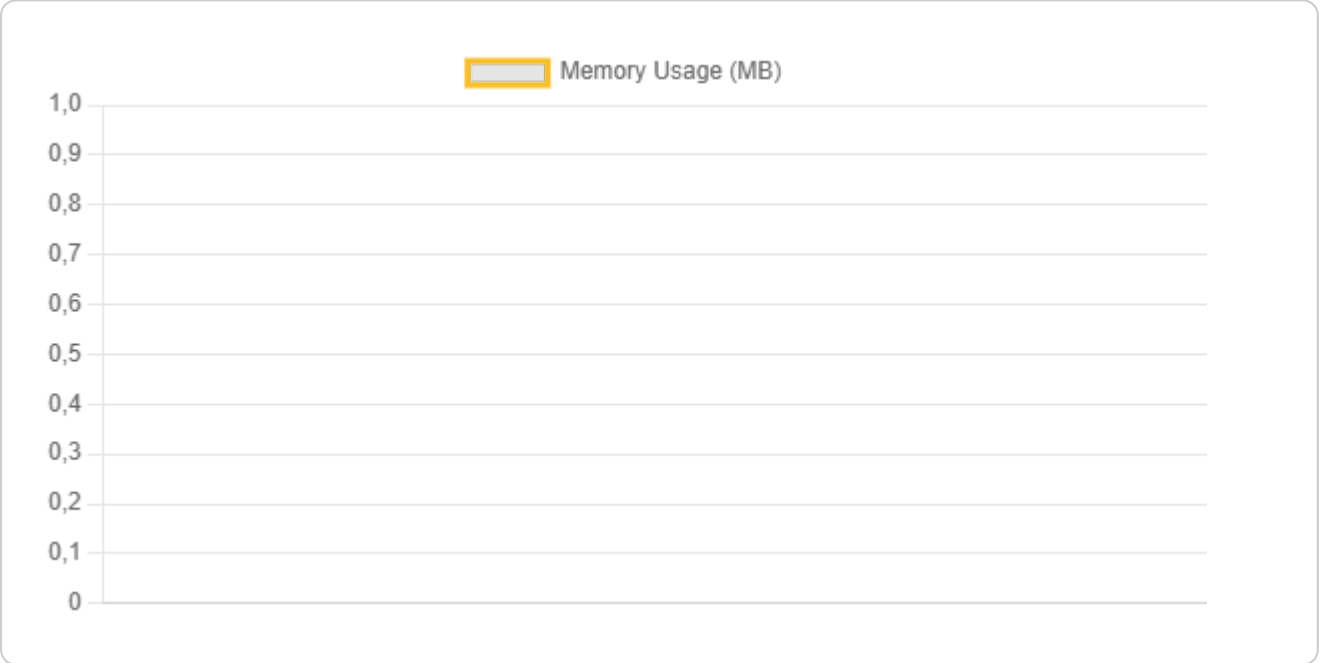


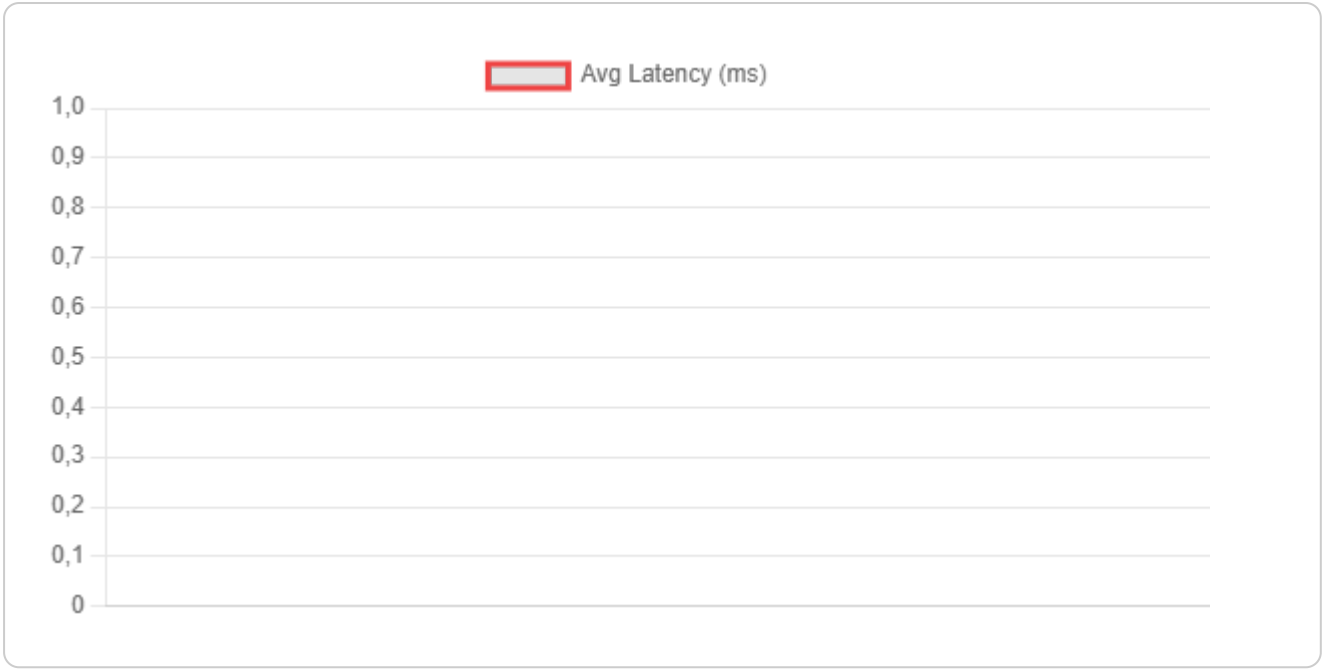



Avg Latency (ms)
1,0
0,9
0,8
0,7
0,6
0,5
0,4
0,3
0,2
0,1
0

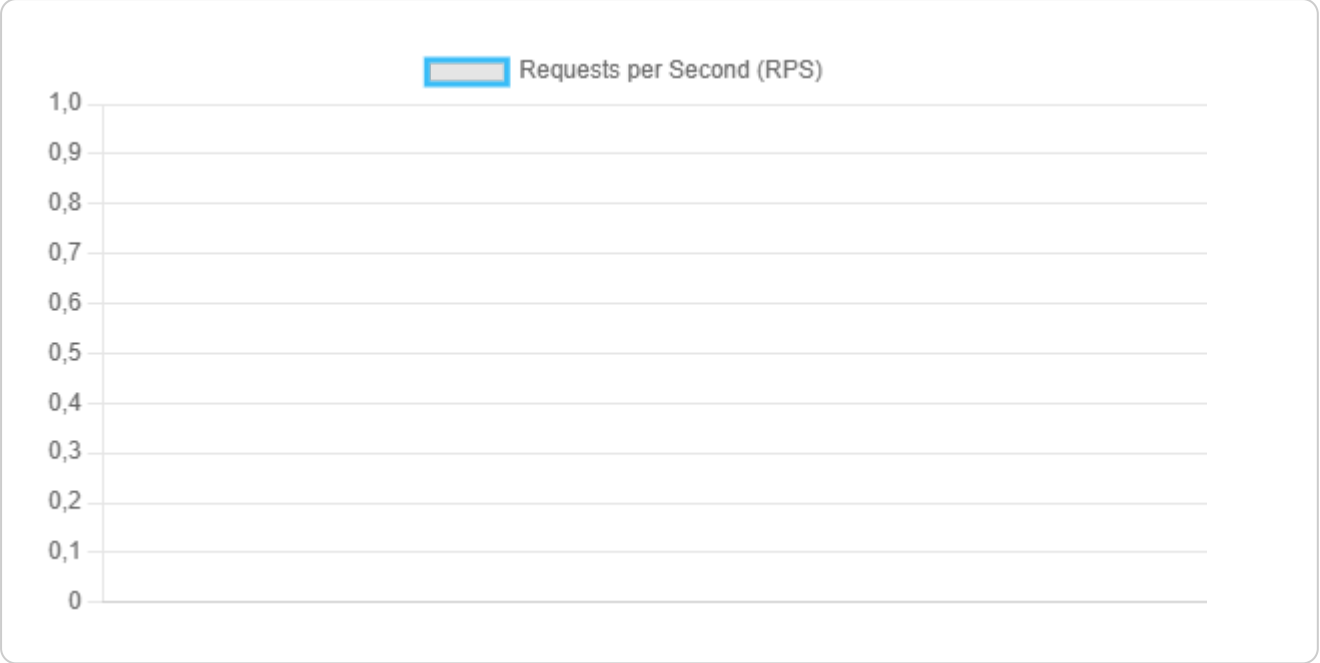
ferrox-rust - Detailed Curves

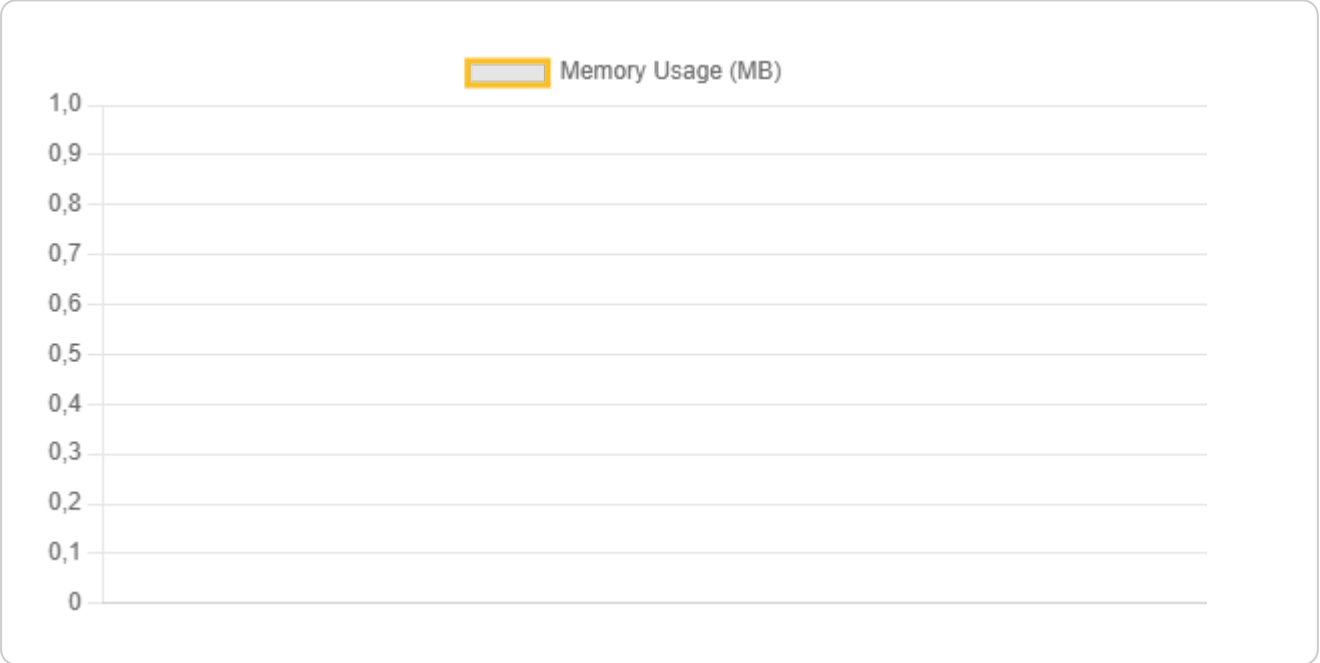


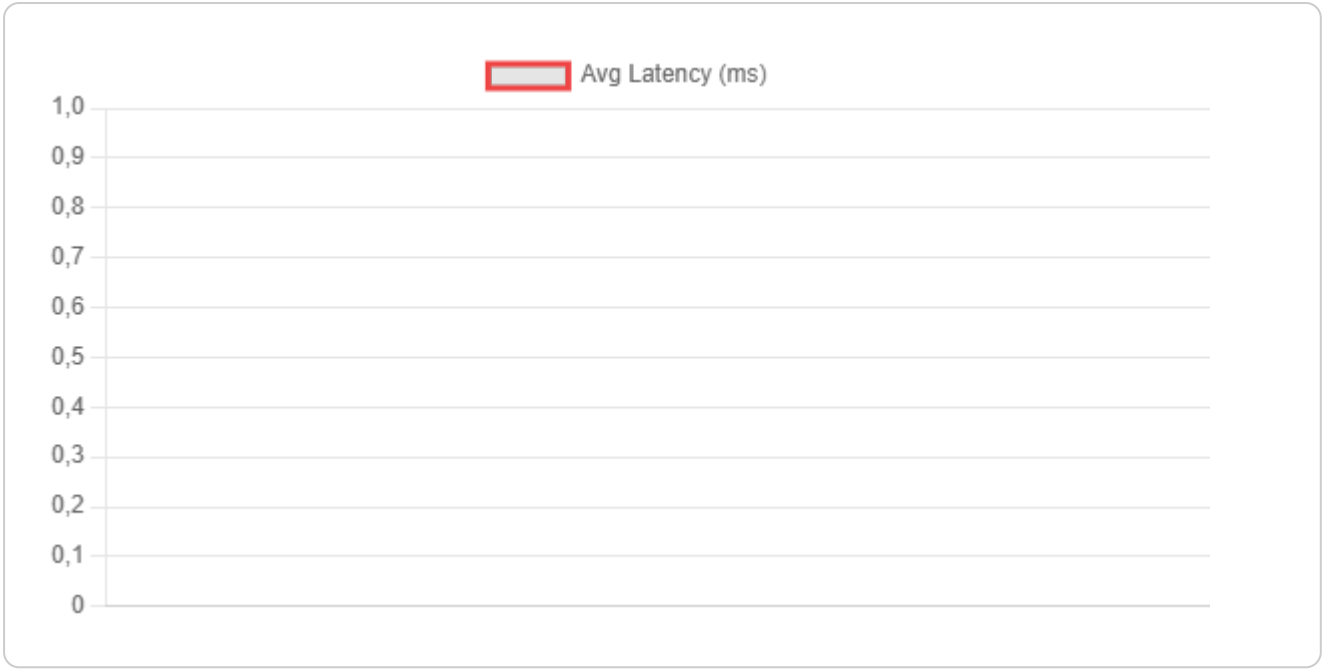



Avg Latency (ms)
1,0
0,9
0,8
0,7
0,6
0,5
0,4
0,3
0,2
0,1
0

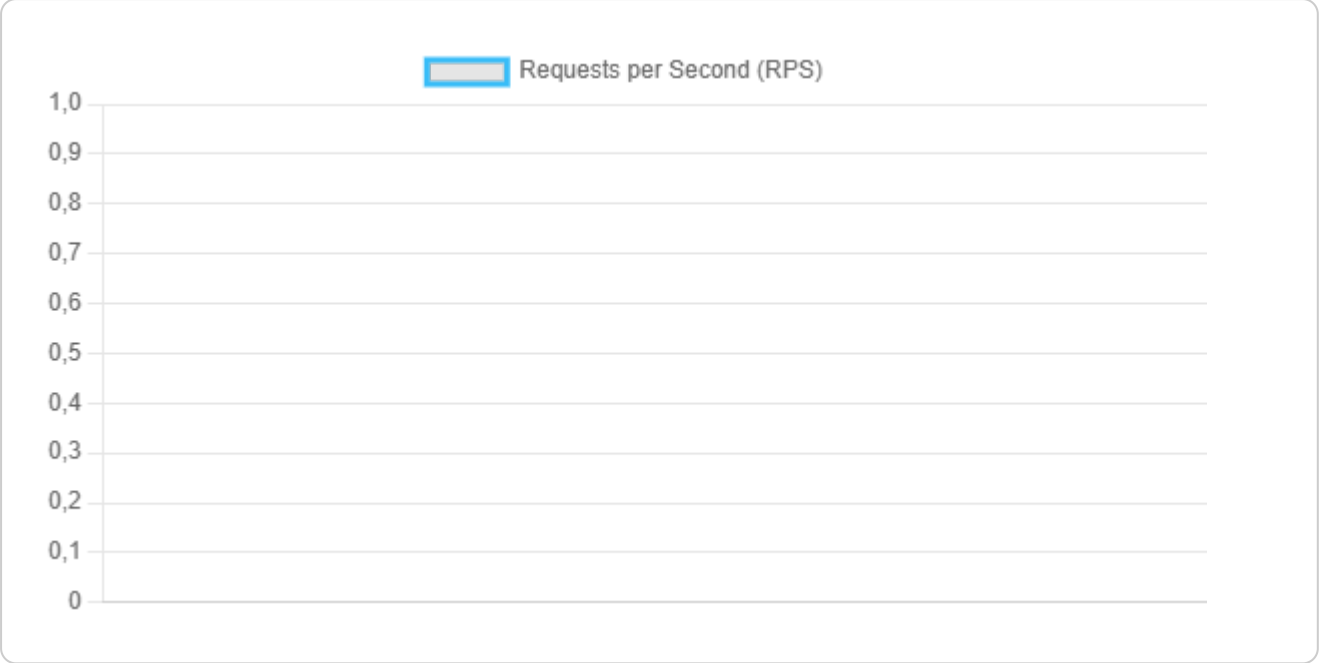
ferrox-java - Detailed Curves

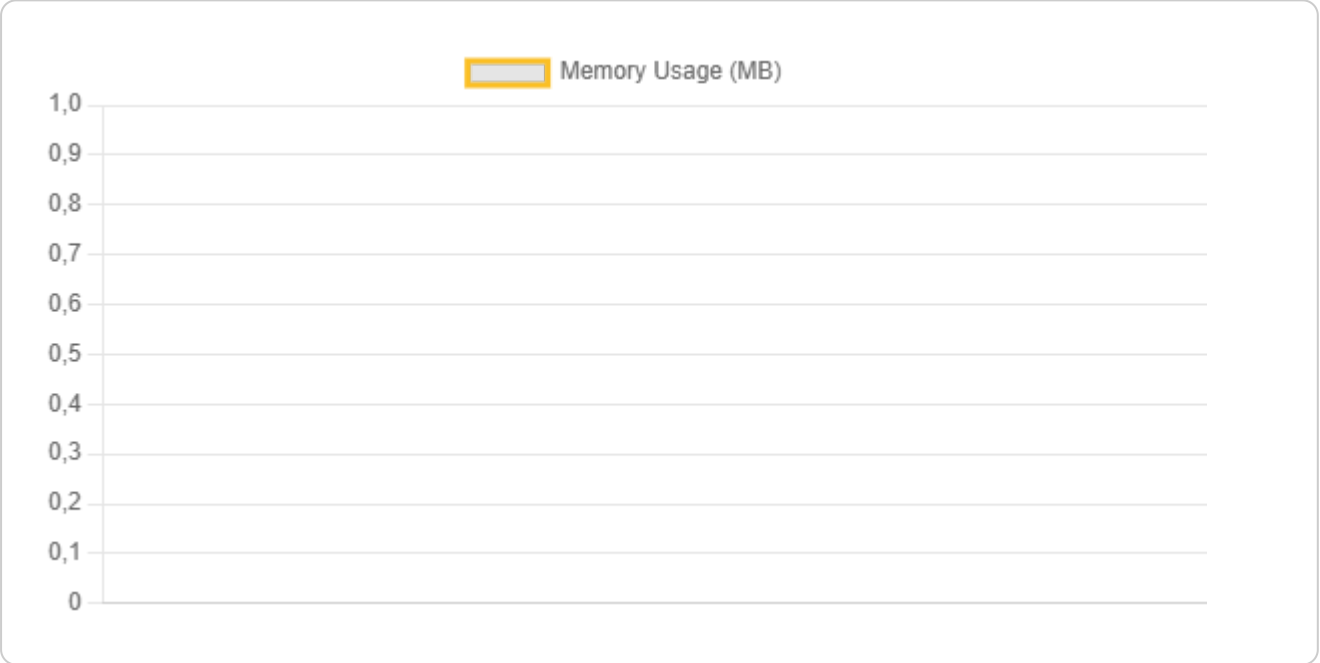


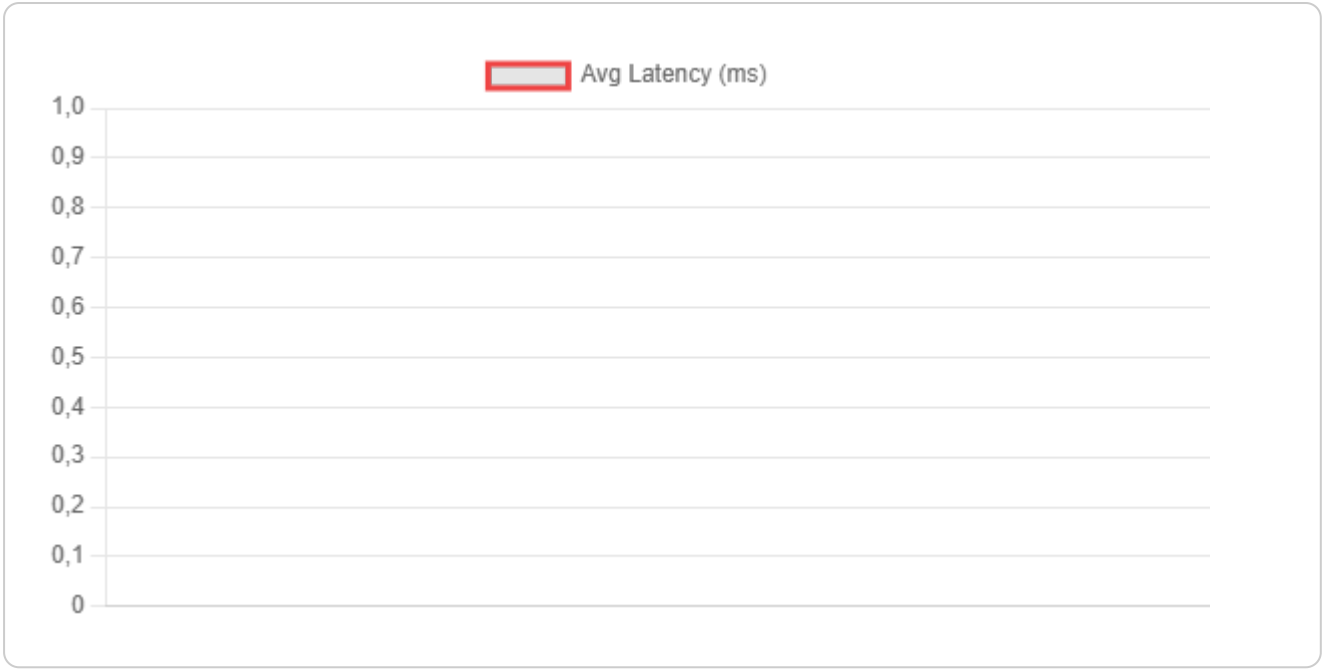



Avg Latency (ms)
1,0
0,9
0,8
0,7
0,6
0,5
0,4
0,3
0,2
0,1
0

ferrox-php - Detailed Curves






Avg Latency (ms)
1,0
0,9
0,8
0,7
0,6
0,5
0,4
0,3
0,2
0,1
0